

Name : Mayank Dudhatra

UID : 24BTCSE100

Batch : B.Tech - CSE - Sem-4

Subject : Advance Java - 030704408

1. Write a simple JDBC program to connect to a MySQL database and print a success message.

Ans :-

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DatabaseConnection {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/your_database_name";
        String user = "your_username";
        String password = "your_password";
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection connection = DriverManager.getConnection(url, user, password);
            if (connection != null) {
                System.out.println("Successfully connected to the MySQL database!");
                connection.close();
            }
        } catch (ClassNotFoundException e) {
            System.out.println("MySQL Driver not found. Please add the Connector/J JAR file.");
            e.printStackTrace();
        } catch (SQLException e) {
            System.out.println("Connection failed! Check your URL, username, or password.");
            e.printStackTrace();
        }
    }
}
```

2. Create a Servlet that prints “Hello from Servlet” on the browser.

Ans :-

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

```

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h1>Hello from Servlet</h1>");
        out.println("</body></html>");
    }
}

```

3. Create a Servlet to read two numbers from the user and display their sum.

Index.html

Ans :- <!DOCTYPE html>

```

<html>
<head>
    <title>Sum Calculator</title>
</head>
<body>
    <form action="SumServlet" method="post">
        Number 1: <input type="number" name="num1"><br><br>
        Number 2: <input type="number" name="num2"><br><br>
        <input type="submit" value="Calculate Sum">
    </form>
</body>
</html>

```

SumServlet.java

```

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/SumServlet")
public class SumServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String n1 = request.getParameter("num1");

```

```

        String n2 = request.getParameter("num2");
        int sum = Integer.parseInt(n1) + Integer.parseInt(n2);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h3>The Sum is: " + sum + "</h3>");
        out.println("</body></html>");
    }
}

```

4. Create a Servlet that demonstrates session handling using HttpSession (store and retrieve a username).

Ans :- index.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Session Login</title>
</head>
<body>
    <form action="SessionServlet" method="post">
        Enter Username: <input type="text" name="username">
        <input type="submit" value="Login">
    </form>
</body>
</html>

```

SessionServlet.java

```

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
@WebServlet("/SessionServlet")
public class SessionServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String name = request.getParameter("username");
        HttpSession session = request.getSession();
        session.setAttribute("user", name);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
    }
}

```

```

        out.println("<html><body>");
        out.println("<h3>Username " + name + " has been saved in session.</h3>");
        out.println("<a href='DisplaySessionServlet'>Go to Welcome Page</a>");
        out.println("</body></html>");
    }
}

DisplaySessionServlet.java
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
@WebServlet("/DisplaySessionServlet")
public class DisplaySessionServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        String name = (String) session.getAttribute("user");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        if (name != null) {
            out.println("<h1>Welcome back, " + name + "!</h1>");
        } else {
            out.println("<h1>No session found.</h1>");
        }
        out.println("</body></html>");
    }
}

```

5. Create a simple JSP page that displays the current date and time.

Ans :- <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
 <%@ page import="java.util.Date" %>
 <!DOCTYPE html>
 <html>
 <head>
 <title>Current Date and Time</title>
 </head>
 <body>

```
<h1>Welcome to JSP Page</h1>
<h2>Current Date and Time is: <%= new Date() %></h2>
</body>
</html>
```

6. Create a JSP page that accepts a name from the user and displays a welcome message.

Ans :- index.jsp

```
<!DOCTYPE html>
<html>
<head>
  <title>Name Input</title>
</head>
<body>
  <form action="welcome.jsp" method="get">
    Enter your name: <input type="text" name="username">
    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

welcome.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
  <title>Welcome Page</title>
</head>
<body>
  <%
    String name = request.getParameter("username");
    if (name != null && !name.trim().isEmpty()) {
  %>
    <h1>Welcome, <%= name %>!</h1>
  <%
    } else {
  %>
    <h1>Welcome, Guest!</h1>
  <%
    }
  %>
  <br>
  <a href="index.jsp">Back</a>
</body>
</html>
```

7. Implement a basic Authentication Filter that allows only requests containing a specific parameter (like a token).

Ans :- AuthFilter.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;

@WebFilter("/SecureServlet")
public class AuthFilter implements Filter {
    public void init(FilterConfig fConfig) throws ServletException {}
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        String token = req.getParameter("token");
        if ("secret123".equals(token)) {
            chain.doFilter(request, response);
        } else {
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            out.println("<h3>Access Denied: Invalid or Missing Token!</h3>");
        }
    }
    public void destroy() {}
}
```

SecureServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/SecureServlet")
public class SecureServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
    }
}
```

```
        PrintWriter out = response.getWriter();
        out.println("<h1>Welcome to the Secure Page!</h1>");
        out.println("<p>Your token was validated successfully.</p>");
    }
}
```

8. Create a Spring Boot project using Spring Initializr and run it successfully.

Ans :- Visit Spring Initializr: Go to start.spring.io.

Project Metadata:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** (Select latest stable version)
- **Dependencies:** Spring Web(for REST APIs) ,and if any needed
- **Group:** `com.example(changeable)`
- **Artifact:** `demo(changeable)`
- **Name:** `demo(changeable)`
- **Packaging:** Jar
- **Java Version:** As per your system

Generate: Click **Generate** to download the ZIP file.

Extract & Open: Extract the ZIP and open the folder in an IDE like VS Code.

Project Structure Overview:

```
demo
├── src/main/java/com/example/demo
│   └── DemoApplication.java
├── src/main/resources
│   └── application.properties
└── pom.xml
```

Run: Locate the main class (annotated with `@SpringBootApplication`) and run it. You should see "Started Application" in the console.

9. Create a REST Controller in Spring Boot that returns a simple “Hello Spring Boot” message.

Ans :-

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello Spring Boot";
    }
}
```

10. Create Spring Boot endpoints for GET and POST operations for a Student resource (without a database).

Ans :- import org.springframework.web.bind.annotation.*;
import java.util.ArrayList;
import java.util.List;
@RestController
@RequestMapping("/students")
public class StudentController {
 private List<String> students = new ArrayList<>();
 @GetMapping
 public List<String> getAllStudents() {
 return students;
 }
 @PostMapping
 public String addStudent(@RequestBody String name) {
 students.add(name);
 return "Student added: " + name;
 }
}

11. Create a Spring Boot service class and call it from a controller to return a processed message. MessageService.java

Ans :- import org.springframework.stereotype.Service;
@Service
public class MessageService {
 public String process(String name) {
 return "Processed Message for: " + name.toUpperCase();
 }
}

MessageController.java

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
@RestController
public class MessageController {
 @Autowired
 private MessageService messageService;
 @GetMapping("/process")
 public String getMessage(@RequestParam String name) {
 return messageService.process(name);
 }
}

12. Connect Spring Boot to MySQL and print “Database Connected Successfully”.

Ans :- application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/your_db_name
spring.datasource.username=root
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update
```

DbCheckRunner.java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
import javax.sql.DataSource;

@Component
public class DbCheckRunner implements CommandLineRunner {
    @Autowired
    private DataSource dataSource;
    @Override
    public void run(String... args) throws Exception {
        if (dataSource.getConnection() != null) {
            System.out.println("Database Connected Successfully");
        }
    }
}
```

13. Create a Student model class and perform basic CRUD operations using Spring Boot + JDBC Template or JPA.

Ans :- Student.java

```
import javax.persistence.*;

@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
}
```

StudentRepository.java

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Long> { }
```

StudentCrudController.java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

```

@RestController
@RequestMapping("/api/students")
public class StudentCrudController {
    @Autowired
    private StudentRepository repo; @GetMapping
    public List<Student> getAll() { return repo.findAll(); }
    @PostMapping
    public Student create(@RequestBody Student s) { return repo.save(s); }
    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) { repo.deleteById(id); }
}

```

14. Test all REST API endpoints using Postman and document request/response outputs.

Ans :- 1. Test Endpoint: Hello Message (Question 9)

- **Method:** GET
- **URL:** <http://localhost:8080/hello>
- **Request Body:** None
- **Response Output:** Hello Spring Boot
- **Status:** 200 OK

2. Test Endpoint: Process Name (Question 11)

- **Method:** GET
- **URL:** <http://localhost:8080/process?name=mayank>
- **Request Body:** None
- **Response Output:** Processed Message for: mayank
- **Status:** 200 OK

3. Test Endpoint: Create Student (Question 13 - POST)

- **Method:** POST
- **URL:** <http://localhost:8080/api/students>
- **Headers:** Content-Type: application/json
- **Request Body (JSON):**

```

{
    "name": "mayank dudhatra",
    "email": "mayank@raiuniversity.edu"
}

```
- **Response Output:**

```

{
    "id": 1,
    "name": "mayank dudhatra",
    "email": "mayank@raiuniversity.edu"
}

```
- **Status:** 201 Created / 200 OK

4. Test Endpoint: Get All Students (Question 13 - GET)

- **Method:** GET
- **URL:** <http://localhost:8080/api/students>
- **Request Body:** None
- **Response Output:**

```
[  
  {  
    "id": 1,  
    "name": "mayank dudhatra",  
    "email": "mayank@raiuniversity.edu"  
  }  
]
```

- **Status:** 200 OK

5. Test Endpoint: Delete Student (Question 13 - DELETE)

- **Method:** DELETE
 - **URL:** <http://localhost:8080/api/students/1>
 - **Request Body:** None
 - **Response Output:** (Empty)
 - **Status:** 200 OK / 204 No Content
-